

Layouts CSS

Contenidos

La propiedad **display**

Layouts Flex

Layouts Grid

Position / Anchor position

Animaciones

Inspirado en el bootcamp de
Manz.dev todos los créditos a el.

La propiedad display

- Los flujos inline se ajustan al contenido
- Los flujos block ocupan el espacio disponible
-  Nueva sintaxis (comportamiento exterior, comportamiento de hijos)

```
.element {  
  display: inline;           /* ✗ Ignora width / height */  
  display: block;            /* ✗ Ignora width / height */  
  display: inline-block;     /* ✔ width / height */  
  display: inline-flex;      /* ✔ width / height */  
  display: flex;             /* ✔ width / height */  
  display: inline-grid;      /* ✔ width / height */  
  display: grid;             /* ✔ width / height */  
}
```

```
.element {  
  display: inline flow;      /* Antiguo inline ✗ Ignora width / height */  
  display: block flow;       /* Antiguo block ✗ Ignora width / height */  
  display: inline flow-root; /* Antiguo inline-block ✗ Ignora width / height */  
  display: inline flex;      /* Antiguo inline-flex ✗ Ignora width / height */  
  display: block flex;       /* Antiguo flex ✗ Ignora width / height */  
  display: inline grid;      /* Antiguo inline-grid ✗ Ignora width / height */  
  display: block grid;       /* Antiguo grid ✗ Ignora width / height */  
}
```

Flex



Flex CSS

- Sistema de una sola dimensión [Flex](#)
- Propiedad flex-direction:
 - row (por defecto)
 - column
 - *-reverse

```
<div class="parent">
  <div class="item">1</div>
  <div class="item">2</div>
  <div class="item">3</div>
  <div class="item">4</div>
  <div class="item">5</div>
  <div class="item">6</div>
</div>
```

```
.parent {
  display: flex;
  flex-direction: row;
  background: #999;

  .item {
    --size: 100px;

    background: indigo;
    border: 5px solid gold;
    color: gold;
    width: var(--size);
    height: var(--size);
  }
}
```

Gaps (Huecos)

- Propiedad **gap** → para añadir huecos entre items (no globales)
- No es lo mismo **gap** (sólo entre items) que **padding** o **margin**

```
.parent {  
  display: flex;  
  flex-direction: row;  
  gap: 1rem;  
  background: #999;  
  
  .item {  
    /* ... */  
  }  
}
```

- Añadimos un **padding** para border exteriores
- Ponemos líneas con **column-rule: 3px solid black**
- Aumentamos el **gap** a **2rem** para dar más espacio
- Ajustamos: **border: 3px solid black** y **width: max-content**
- ⚠ Ajustamos las líneas con **column-rule-inset: -1rem**
Pro-tip:
- Añadimos un **--offset: 2rem**
- Mutamos **column-rule-inset: calc(var(--offset) * -1)**
- Mutamos **padding: var(--offset)**
- Mutamos **gap: calc(var(--offset) * 2)**

Variables CSS

- Variable definida en el elemento
- Variable definida en el padre
- Variable definida en el HTML
- Uso de fallbacks
- Ámbitos de variables de CSS

```
.element {  
  --size: 50px;  
  
  width: var(--size);          /* No usa fallback */  
  height: var(--size, 50px); /* Usa fallback */  
}
```

Flex multilinea (Wrap)

- Se dice que Flex funciona con una dimensión, pero...
- Propiedad **flex-wrap*** → con **wrap** permite multilinea (desborda items sin deformarlos)

```
<div class="parent">
  <div class="item">1</div>
  <div class="item">2</div>
  <div class="item">3</div>
  <div class="item">4</div>
  <div class="item">5</div>
  <!-- ... hasta 10 -->
</div>
```

```
.parent {
  display: flex;
  flex-direction: row;
  flex-wrap: wrap;
  justify-content: space-between;
  align-content: space-between;
  background: grey;
  padding: 1rem;
  width: 600px;
  gap: 1rem;
}
```

- El modo **wrap** desbloquea **align-content** para alinear esas líneas extra.

Extras de Flex

- Con **justify-content** → eje primario (row → horizontal, column → vertical)
- Con **align-items** → eje secundario (row → vertical, column → horizontal)
- En modo wrap, **align-content** permite alinear eje extra multilinea
- Extra: El maravilloso **order**



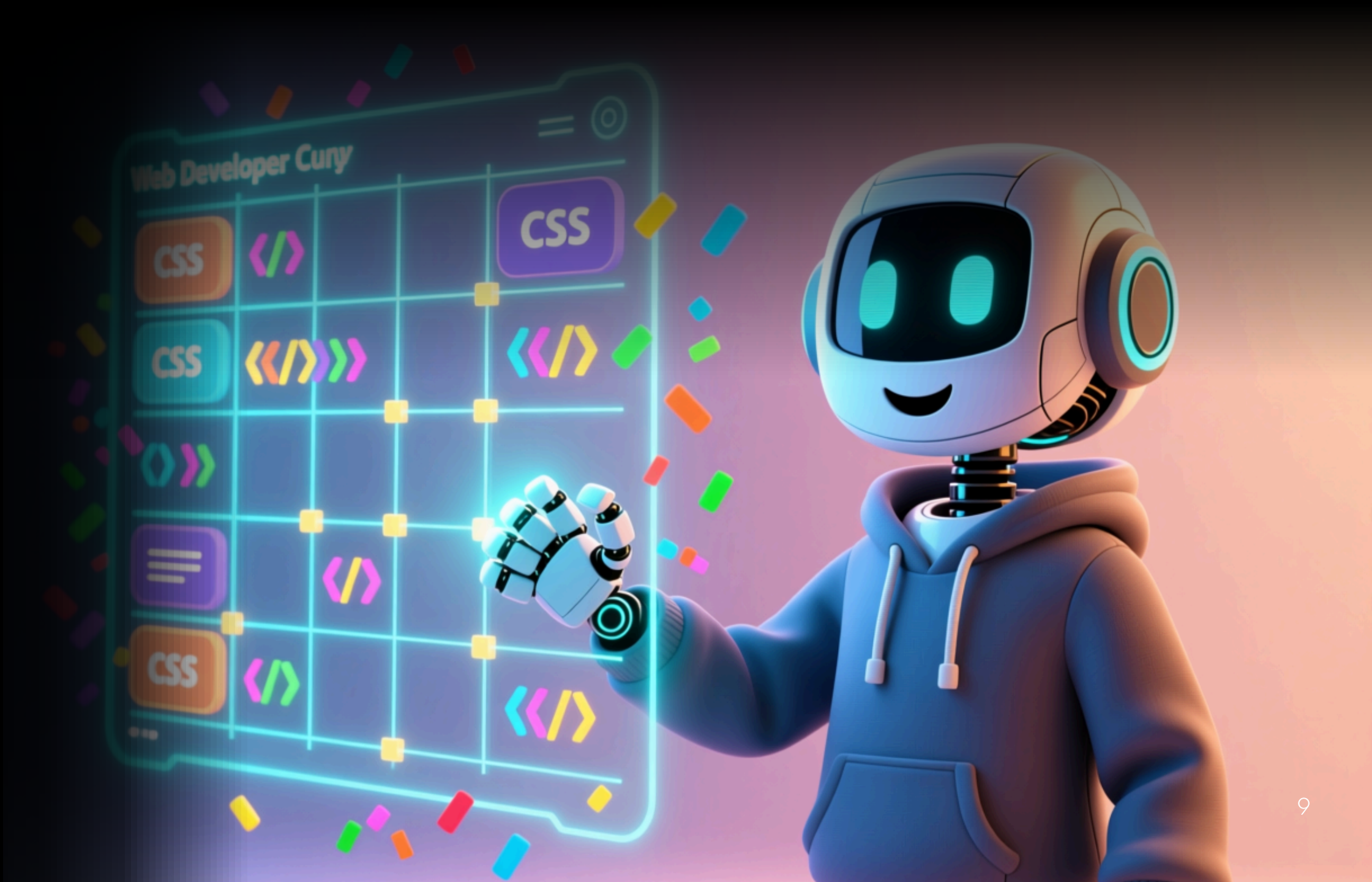
```
.cat { order: 0 }
```

Juegos para practicar:

[FlexboxFroggy](#)

[FlexboxDefense](#)

Grid



Grid

- Sistema de dos dimensiones (cuadrículas) → Conceptos: [Grid](#)
- Definir el tamaño de ancho → grid-template-columns
- Definir el tamaño de alto → grid-template-rows
- Definir huecos con gap → (row + columns)

```
<div class="grid">
  <div class="item">1</div>
  <div class="item">2</div>
  <div class="item">3</div>
  <div class="item">4</div>
  <div class="item">5</div>
  <div class="item">6</div>
</div>
```

```
.grid {
  display: grid;
  grid-template-columns: 100px 100px 100px;
  grid-template-rows: 50px 50px;
  gap: 15px 15px;
  width: max-content;
  background: grey;

  .item { background: indigo }
}
```

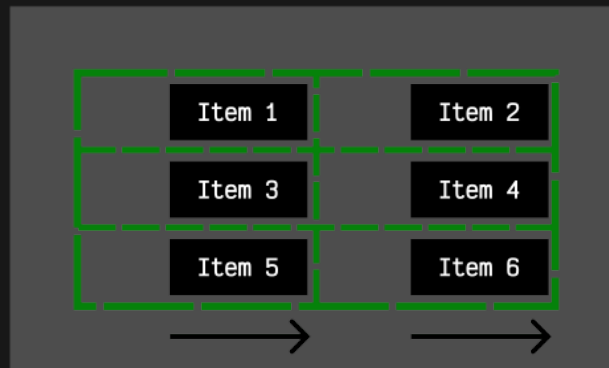
Más características en Grid

- La función `repeat(num, size)` ayuda a simplificar
- La unidad `fr` (fracción restante)
- Responsive one-line: `repeat(auto-fill, minmax(var(--size), 1fr))`

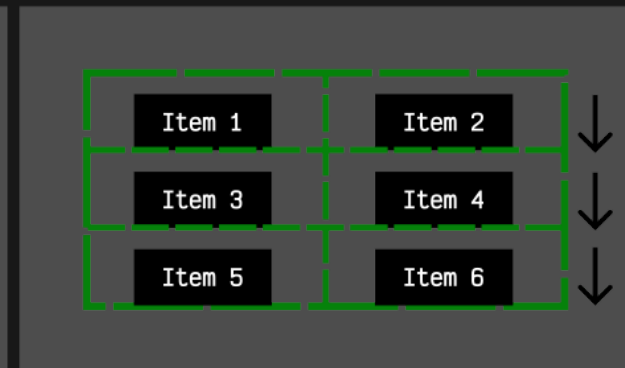
```
.grid {  
  display: grid;  
  grid-template-columns: repeat(3, 100px) 200px;  
  grid-template-rows: repeat(2, 50px);  
  gap: 15px 15px;  
  width: max-content;  
  background: grey;  
  
  .item { background: indigo }  
}
```

Extras de Grid

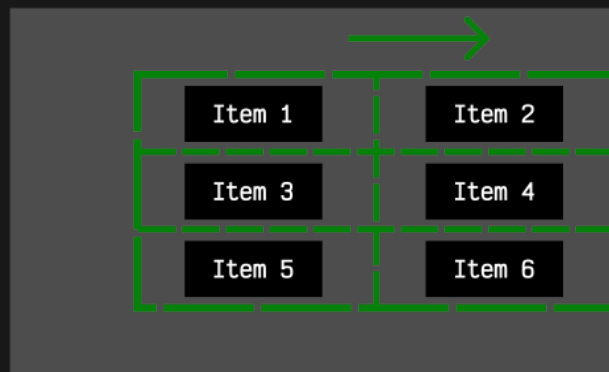
- Con las propiedades `*-items` y `*-content` se puede alinear elementos
- Con `justify-items` y `align-items` (hijos, dentro de celdas)
- Con `justify-content` y `align-content` (el propio grid)
- Más info → [Alinear en Grid](#)



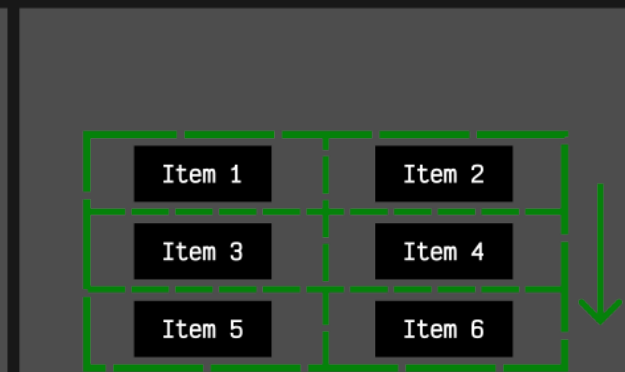
justify-items: end



align-items: end



justify-content: end



align-content: end

Grid por áreas

- Grid por áreas: `grid-template-areas` y `grid-area`
- Zonas vacías con `.` (asegúrate de tener mismo número de columnas)

```
<div class="parent">
  <header>Logo</header>
  <section>Content</section>
  <aside>Menú</aside>
  <footer>Footer</footer>
</div>
```

```
.parent {
  display: grid;
  grid-template-areas: "head head"
                      "body menu"
                      "foot foot";
  grid-template-rows: 100px 1fr 100px;
  height: 700px;
}

header { grid-area: head; background: blue }
section { grid-area: body; background: green }
aside { grid-area: menu; background: grey }
footer { grid-area: foot; background: #333 }
```

Grids irregulares

- Grids irregulares: `grid-*-start` y `grid-*-end`
- Zonas vacías con `.` (asegúrate de tener mismo número de columnas)

```
<div class="parent">
  <div class="item">1</div>
  <div class="item">2</div>
  <div class="item">3</div>
  <div class="item">4</div>
  <div class="item">5</div>
</div>
```

```
.parent {
  display: grid;
  grid-template-columns: repeat(2, 100px);
  grid-template-rows: 50px 1fr 100px;
  gap: 5px;
}

.item { background: grey }

.item:nth-child(2) {
  grid-row-start: 2; /* +1 */
  grid-column: 1 / span 2; /* De 1 hasta 2 */
}
```

Juegos para practicar:

- [Grid Garden](#)

Position



Posicionamiento CSS

- La propiedad **position**
- 5 modalidades: **static**, **relative** y **absolute** (variaciones: **fixed** y **sticky**)
- Desplazamientos: **top**, **bottom**, **left**, **right**

```
.item-1 {  
  position: relative;  
  left: 5px;           /* → 5px (desde la izquierda) */  
  left: -5px;          /* ← 5px (hacia la izquierda) */  
  top: 5px;            /* ↓ 5px (hacia abajo) */  
  top: -5px;           /* ↑ 5px (hacia arriba) */  
}
```

```
.parent {  
  position: relative; /* Marco (referencia) */  
  
  .item {  
    position: absolute; /* Busca padre != static */  
    left: 0;  
    top: 0;  
  }  
}
```

Posicionamiento CSS

- La propiedad **position**
- 5 modalidades: **static**, **relative** y **absolute** (variaciones: **fixed** y **sticky**)
- Desplazamientos: **top**, **bottom**, **left**, **right**

```
.parent {  
  .box {  
    background: indigo;  
    position: fixed;  
    width: 50px;  
    height: 50px;  
    right: 0;  
    top: 0;  
  }  
}
```

```
.parent {  
  .box {  
    background: indigo;  
    position: sticky;  
    top: 0;  
    height: 50px;  
  }  
}
```

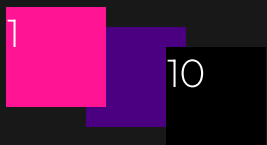
Profundidad CSS

- La propiedad **z-index** (valores numéricos)

```
<div class="container">
  <div class="item item-1"></div>
  <div class="item item-2"></div>
  <div class="item item-3"></div>
</div>
```

```
.item {
  width: 100px;
  height: 100px;
  position: relative;

  &.item-1 { background: deeppink }
  &.item-2 { background: indigo; top: -40px; left: 40px }
  &.item-3 { background: black; top: -80px; left: 80px }
  &.item-1 { z-index: 1 }
  &.item-2 { z-index: var(--item-index, 0) }
  &.item-3 { z-index: 10 }
}
```



```
.item-2 { z-index: 5 }
```

CSS Anchor position

```
<div class="reference">Reference</div>
<div class="element">Element</div>

<style>
.reference { anchor-name: --ref }

.element {
  position: absolute;
  position-anchor: --ref;
  bottom: anchor(bottom); 1
  right: anchor(left);    2
}
</style>
```

- 1 Ancla el borde inferior del elemento a la referencia
- 2 Ancla el borde derecho del elemento a la referencia

ElementReference

Animaciones

La propiedad `transition`

La regla `@starting-style`

Animaciones con `@keyframes`

Ritmo (funciones de tiempo)

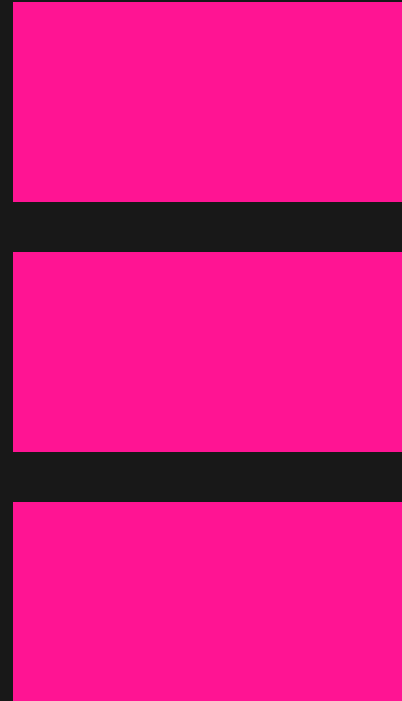
Animaciones de scroll



Transiciones

- La palabra clave all aplica a todas las propiedades
- Transiciones de entrada y de salida

```
.element {  
  background: deeppink;  
  width: 200px;  
  transition: all 0.5s;  
}  
  
.element:hover {  
  width: 300px;  
  background: indigo;  
}
```



La regla `@starting-style`

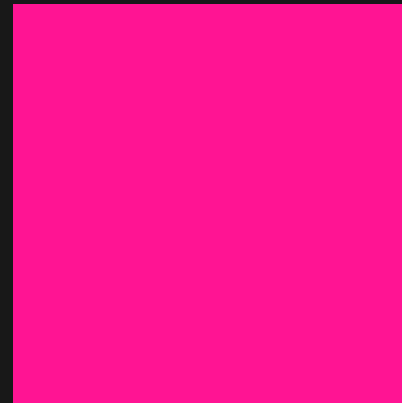
- Estilo inicial (útil para evitar saltos en entradas)
- Se puede aplicar `nesting` en los estilos

```
.element {  
  background: deeppink;  
  width: 200px;  
  height: 75px;  
  transition: all 0.75s;  
  opacity: 1;  
}  
  
@starting-style {  
  .element { opacity: 0 }  
}
```


Animaciones

- Con `@keyframes` creamos la animación
- Con `animation` la aplicamos → Más opciones de animación

```
.element {  
  background: deeppink;  
  width: 200px;  
  height: 200px;  
  animation: move 2s alternate infinite;  
}  
  
@keyframes move {  
  0%, 100% { transform: translate(0, 0); }  
  50% { transform: translate(200px, 0); }  
}
```



Ritmo (funciones de tiempo)

- Constante (mismo ritmo): `linear`
- Ritmos variables: `ease` (por defecto), `ease-in`, `ease-out`, `ease-in-out`
- Ritmo personalizado: función `cubic-bezier()`
- Ritmo lineal personalizado: función `linear()`
- Ritmos escalonados: función `steps()` → Ejemplo con SpriteSheets

Animaciones de scroll

- La función view() → [Ejemplo](#)

```
.element {  
  background: indigo;  
  animation: change linear both;  
  animation-timeline: view(block 20%);  
}
```

```
@keyframes change {  
  from { scale: 0; opacity: 0 }  
  to { scale: 1; opacity: 1; }  
}
```

View Transition

- ¿Qué son las View Transition?
- Recomendable en un `<style>` inline (performance)

```
/* Obligatorio */
@view-transition { navigation: auto }

/* Opcional */
.container {
  view-transition-name: page;
}

::view-transition-old(page) { animation: fade 0.2s linear forwards }
::view-transition-new(page) { animation: fade 0.3s linear reverse }

@keyframes fade {
  from { opacity: 1 }
  to { opacity: 0 }
}
```

Referencias

- [CheatSheet CSS](#)
- [bootcamp.manz.dev](#)